WEB DEVELOPMENT • TERRALOGIC ACADEMY • BATCH 3 (MERN STACK)

# HTML & CSS

Every HTML tag explained in detail, and how to use CSS efficiently to style, layout, and polish a real web page.

*Document structure • Every major HTML tag • CSS selectors & box model • Efficient styling*

TODAY'S SESSION

# What We'll Cover

### 1. What is HTML & Document Anatomy

How a browser reads HTML, CSS and JS together

### 2. Every Major HTML Tag

Structure, text, lists, links, media, tables & forms

### 3. Semantic HTML5 & Attributes

header/nav/main/footer, id, class, data-*

### 4. Local Dev Server

Recap: serving your page with npx http-server

### 5. Introducing CSS

What CSS is, and the 3 ways to apply it

### 6. Selectors & The Box Model

How CSS targets elements and measures space

### 7. Core Properties & Flexbox

Color, typography, spacing, display & layout

### 8. Best Practices & What's Next

Writing efficient CSS, then previewing our project

SECTION 1

# Understanding HTML

What HTML actually is, and how a browser turns it into the page you see.

CONCEPT

# What is HTML, and What Does It Do?

### HTML = HyperText Markup Language

HTML isn't a programming language - it's a markup language. It uses tags to describe the structure and meaning of content: "this is a heading", "this is a list", "this is a button".

**How a browser builds the page you see:**

- Browser downloads the .html file
- It parses tags into a tree structure (the DOM)
- CSS rules are applied to style that tree
- JavaScript can then change it after load
- The final result is painted on screen

### HTML — the skeleton

Defines structure and content: headings, paragraphs, images, forms.

### CSS — the skin

Defines appearance: colors, fonts, spacing, layout, responsiveness.

### JavaScript — the behaviour

Defines interactivity: clicks, calculations, dynamic updates (later classes).

THE STARTING POINT

# Anatomy of an HTML Document

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>My Page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first page.</p>
  </body>
</html>
```

**<!DOCTYPE html>**

Tells the browser this is an HTML5 document. Always the very first line.

**<html lang="en">**

The root element. lang helps screen readers & translators.

**<head>**

Invisible metadata: title, character set, linked CSS.

**<body>**

Everything inside here is visible to the user.

SECTION 2

# Core Structure & Text Tags

The foundational tags every single HTML page is built from.

TAG REFERENCE

# Document Structure Tags

## <!DOCTYPE html>

Declares the document as HTML5. Must be the very first line. It is not a tag pair - there is no closing version.

**e.g.** *<!DOCTYPE html>*

## <html>

The root element that wraps the entire page. Every other tag lives inside it.

**e.g.** *<html lang="en">…</html>*

## <head>

Holds metadata that is not displayed: title, character set, linked stylesheets, SEO tags.

**e.g.** *<head>…</head>*

## <title>

Sets the text shown in the browser tab, and used by search engines as the clickable result title.

**e.g.** *<title>My Page</title>*

## <meta>

Self-closing metadata tag - character encoding, page description, and the viewport tag for responsive design.

**e.g.** *<meta name="viewport" content="width=device-width">*

## <body>

Contains everything visible on the page - text, images, links, forms. The part users actually see.

**e.g.** *<body>…</body>*

TAG REFERENCE

# Headings & Containers

## <h1> ... <h6>

Six levels of headings, h1 the most important. Use exactly one h1 per page; never skip levels just to change size - use CSS for that.

**e.g.** *<h1>Page Title</h1>*

## 

A paragraph of text. Browsers automatically add space above and below it.

**e.g.** * Some text here. *

## 

A generic block-level container with no meaning of its own - used to group elements for layout and styling.

**e.g.** * ... *

## 

A generic inline container - styles a small piece of text inside a sentence without breaking the line.

**e.g.** *word*

## <br>

Forces a single line break inside text. Self-closing, has no content or closing tag.

**e.g.** *Line one<br>Line two*

## <hr>

Draws a horizontal divider line, used to visually separate sections of content.

**e.g.** *<hr>*

TAG REFERENCE

# Text Formatting: Emphasis & Highlight

### 

Marks text as important. Bold by default, and carries real semantic meaning for screen readers.

**e.g.** *Warning*

### 

Bold text with no extra importance - purely visual. Prefer  when the meaning actually matters.

**e.g.** *Bold word*

### 

Marks stress emphasis. Italic by default, and read with emphasis by screen readers.

**e.g.** *really means it*

### 

Italic text with no extra meaning - e.g. foreign words, technical terms, or ship/book names.

**e.g.** *The Hindu*

### <mark>

Highlights text with a yellow background by default, like a highlighter pen - great for search-result matches.

**e.g.** *<mark>matched text</mark>*

### <sub> / <sup>

Subscript and superscript text, used for chemical formulas and exponents.

**e.g.** *H<sub>2</sub>O, x<sup>2</sup>*

TAG REFERENCE

# Text Formatting: Quotes & Code

## <blockquote>

A long, indented quotation from another source. Browsers add margin automatically.

**e.g.** *<blockquote>…</blockquote>*

## <q>

A short inline quotation - the browser automatically adds quotation marks around it.

**e.g.** *She said <q>hello</q>.*

## <pre>

Preformatted text - preserves every space and line break exactly as typed, shown in a monospace font.

**e.g.** *<pre> line 1\n line 2</pre>*

## <code>

Marks inline programming code, like a variable name, file name, or terminal command.

**e.g.** *Run <code>npm install</code>*

## <kbd>

Represents user keyboard input - useful in tutorials and instructions.

**e.g.** *Press <kbd>Ctrl+C</kbd>*

## <abbr>

Marks an abbreviation; the title attribute shows the full expansion on hover.

**e.g.** *<abbr title="HyperText…">HTML</abbr>*

TAG REFERENCE

# List Tags

## <ul>

Unordered (bulleted) list - use when the order of items doesn't matter.

**e.g.** *<ul><li>Tea</li></ul>*

## <ol>

Ordered (numbered) list - use when sequence matters, like step-by-step instructions.

**e.g.** *<ol><li>Step one</li></ol>*

## <li>

A single list item. Must always live inside a <ul> or <ol> parent.

**e.g.** *<li>Item text</li>*

## <dl>

Description list - a wrapper for pairs of terms and their definitions. Great for glossaries and FAQs.

**e.g.** *<dl>...</dl>*

## <dt>

The term being defined, inside a <dl>.

**e.g.** *<dt>HTML</dt>*

## <dd>

The definition/description for the preceding <dt>, inside a <dl>.

**e.g.** *<dd>Markup language</dd>*

SECTION 3

# Links, Media & Tables

Connecting pages together and embedding images, video, and tabular data.

# Link & Navigation Tags

## <a href="">

Creates a hyperlink to another page, file, or location. href is the destination - the most important attribute.

**e.g.** *<a href="about.html">About</a>*

## target="_blank"

Opens the link in a new browser tab instead of replacing the current page.

**e.g.** *<a href="..." target="_blank">*

## rel="noopener noreferrer"

A security best practice - always pair with target="_blank" so the new tab can't access your page's window object.

**e.g.** *rel="noopener noreferrer"*

## <a href="#id">

Jumps to the element with a matching id on the same page - used for in-page navigation like 'Back to top'.

**e.g.** *<a href="#contact">Contact</a>*

## <nav>

A semantic wrapper for a group of navigation links - typically the main site menu. Helps browsers, search engines, and screen readers understand 'this block is for navigating the site'.

**e.g.** *<nav><a>Home</a><a>About</a></nav>*

# Image Tags & Figures

## <img src="" alt="">

Embeds an image. src is the file path or URL. alt is REQUIRED for accessibility - it's read aloud by screen readers and shown if the image fails to load.

**e.g.** *<img src="logo.png" alt="Company logo">*

## width / height

Attributes that reserve space for the image before it finishes loading, which prevents the rest of the page from jumping around (layout shift).

**e.g.** *<img src="logo.png" width="200" height="80" alt="Logo">*

## <figure>

Wraps an image (or chart, diagram, code listing) together with its caption as one semantic, self-contained unit.

**e.g.** *<figure>...</figure>*

## <figcaption>

The caption describing the figure's content. Always placed inside <figure>, usually right after the <img>.

**e.g.** *<figcaption>Fig 1. Our team</figcaption>*

TAG REFERENCE

# Audio, Video & iFrame Tags

## <video controls>

Embeds a video with built-in play, pause, volume and fullscreen controls. Add src or use nested <source> tags for multiple formats. Use width to control its size.

**e.g.** *<video src="demo.mp4" controls width="480"></video>*

## <audio controls>

Embeds a sound clip with playback controls - play/pause and a volume slider. Same src/source pattern as <video>.

**e.g.** *<audio src="clip.mp3" controls></audio>*

## <iframe src="">

Embeds an entire separate web page inside the current one - commonly used for an embedded YouTube video, Google Map, or payment widget.

**e.g.** *<iframe src="https://maps..."></iframe>*

TAG REFERENCE

# Table Tags - The Basics

## <table>

The container for an entire table. Everything else - rows, headers, cells - lives inside it.

**e.g.** *<table>...</table>*

## <tr>

Table Row - represents one horizontal line of cells. A table is simply a stack of <tr> rows.

**e.g.** *<tr><td>...</td></tr>*

## <th>

Table Header cell - bold and centered by default, describes the column or row beneath/beside it.

**e.g.** *<th>Name</th>*

## <td>

Table Data cell - holds one piece of actual table content, inside a <tr>.

**e.g.** *<td>Satya</td>*

TAG REFERENCE

# Table Tags - Grouping & Spanning

## <thead> / <tbody> / <tfoot>

Group the header row(s), the main data rows, and footer row(s) (e.g. totals) of a table respectively.

**e.g.** *<thead><tr><th>...</th></tr></thead>*

## colspan="2"

Makes one cell stretch across multiple columns - e.g. a title row spanning the whole table width.

**e.g.** *<td colspan="2">Subtotal</td>*

## rowspan="2"

Makes one cell stretch down across multiple rows - e.g. a category label shared by several rows.

**e.g.** *<td rowspan="2">Books</td>*

**Example: a simple table**

| Task | Hours | Rate |
|---|---|---|
| UI Design | 8 | $40 |
| Backend API | 20 | $45 |

```
<table>
  <thead><tr><th>Task</th>...</tr></thead>
  <tbody><tr><td>UI Design</td>...</tr></tbody>
</table>
```

SECTION 4

# Forms & Semantic HTML

Collecting input from users, and structuring pages so both browsers and people understand them.

TAG REFERENCE

# The <form> Tag & Input Types

## <form action="" method="">

Wraps every form control on the page. action is the URL the data is sent to; method (GET or POST) defines how it's sent. Everything below lives inside one <form>.

**e.g.** *<form action="/submit" method="POST">...</form>*

| type= | What it does | Example |
|---|---|---|
| "text" | A single-line free text field | <input type="text" name="fullname"> |
| "email" | Validates the value looks like an email address | <input type="email" name="email"> |
| "password" | Masks each typed character with a dot | <input type="password" name="pwd"> |
| "number" | Only accepts numeric input, with up/down arrows | <input type="number" name="hours"> |
| "checkbox" | A box the user can tick on/off - multiple can be selected | <input type="checkbox" name="agree"> |
| "radio" | One choice from a group sharing the same name attribute | <input type="radio" name="plan"> |
| "date" | Opens a native date picker | <input type="date" name="dob"> |
| "file" | Lets the user choose a file to upload | <input type="file" name="resume"> |
| "submit" | A button that submits the whole form | <input type="submit" value="Send"> |

TAG REFERENCE

# More Form Tags

## <label for="id">

The text caption tied to one specific input via matching id/for. Clicking the label focuses or checks that input - essential for accessibility.

**e.g.** *<label for="email">Email</label>*

## <textarea>

A multi-line text input box - for comments, messages, descriptions. Resizable by the user by default.

**e.g.** *<textarea rows="4"></textarea>*

## <select> / <option>

A dropdown list. <select> is the container; each <option> inside it is one selectable choice.

**e.g.** *<select><option>Small</option></select>*

## <button>

A clickable button. type="submit" submits the form; type="button" does nothing on its own (used with JavaScript later).

**e.g.** *<button type="submit">Send</button>*

## <fieldset> / <legend>

<fieldset> groups related form fields together visually and semantically; <legend> gives that group a caption.

**e.g.** *<fieldset><legend>Contact</legend>...</fieldset>*

ATTRIBUTE REFERENCE

# Key Form Attributes

| Attribute | What it does | Example |
|---|---|---|
| `placeholder` | Greyed-out hint text shown until the user starts typing | `placeholder="you@email.com"` |
| `required` | Browser blocks form submission until this field has a value | `required` |
| `value` | Sets the default or pre-filled value of a field | `value="0"` |
| `name` | The key used to identify this field's data when the form is submitted | `name="email"` |
| `maxlength / min / max` | Limits the number of characters, or the numeric range, allowed | `maxlength="20"  min="1" max="100"` |
| `disabled / readonly` | disabled removes the field from submission entirely; readonly shows it but blocks editing | `disabled` |

PAGE STRUCTURE

# Semantic HTML5 Layout Tags

Semantic tags describe the ROLE of a section, not just its box - this helps SEO, accessibility, and other developers reading your code.

**<header>** Introductory content - usually a logo, site title and/or navigation.

**<nav>** A block of navigation links (the main menu, or a sidebar of links).

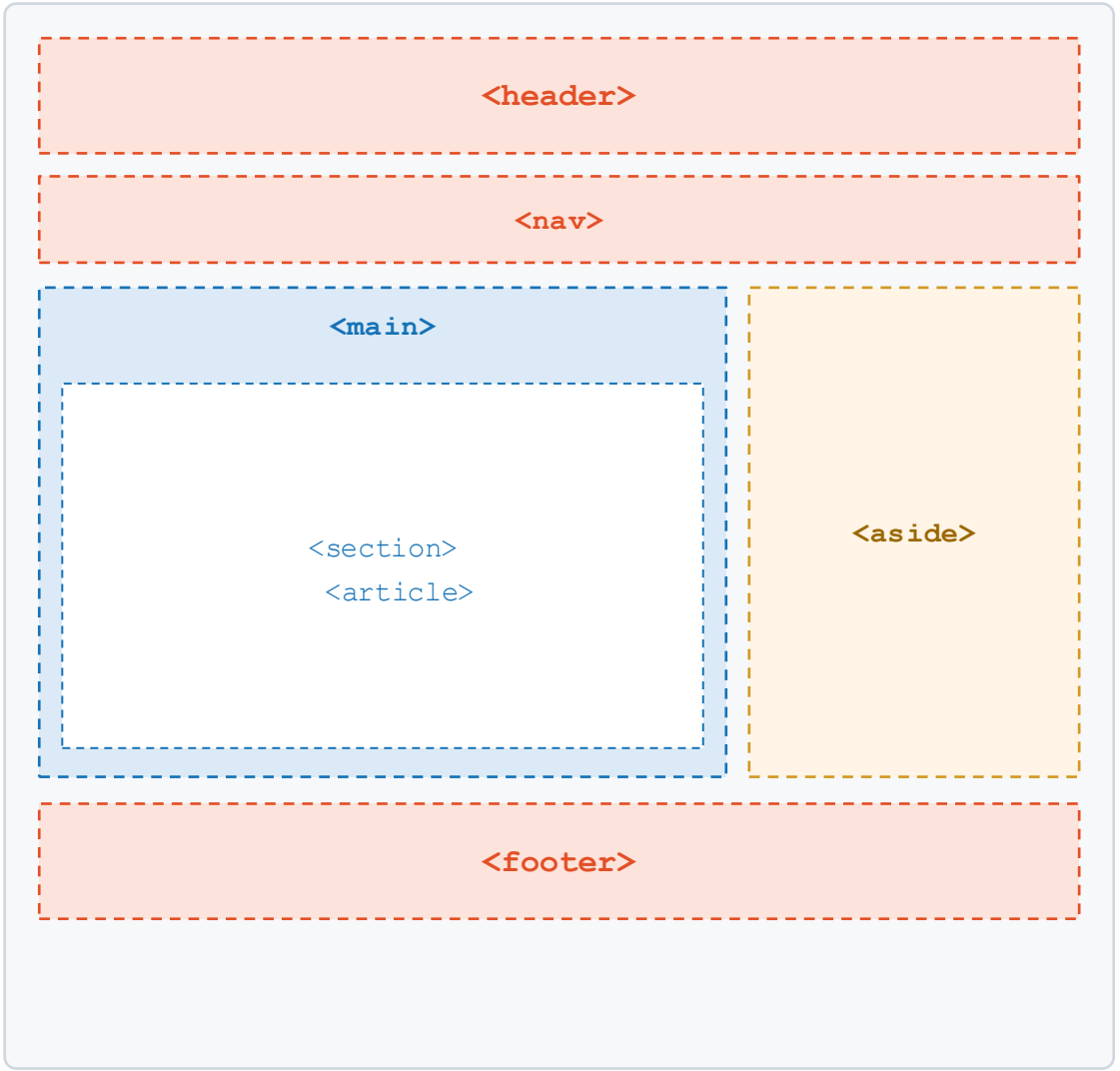
**<main>** The single, central content unique to this page. Only one per page.

**<section>** A thematic grouping of related content, usually with its own heading.

**<article>** Self-contained content that could stand alone - a blog post, news story.

**<aside>** Content tangentially related to the main content - sidebars, pull quotes.

**<footer>** Closing content - copyright, contact info, footer links.



ATTRIBUTE REFERENCE

# HTML Global Attributes

Global attributes can be added to almost any HTML tag, regardless of what it is.

### `id="..."`

A unique identifier for exactly ONE element on the page - used by CSS or JS to target it specifically.

**e.g.** * *

### `class="..."`

A reusable label that can be applied to MANY elements - used by CSS to style a whole group consistently.

**e.g.** *<p class="note">*

### `style="..."`

Inline CSS applied directly to one element. Quick for testing, but avoid in real projects - use external CSS instead.

**e.g.** *style="color: red;"*

### `title="..."`

Tooltip text shown when the user hovers over the element.

**e.g.** *title="Click to expand"*

### `data-*`

Custom data attributes for storing extra information your own JavaScript can read later.

**e.g.** *data-user-id="42"*

### `lang="..."`

Declares the language of the content - important for accessibility tools and translators.

**e.g.** *lang="en"*

QUICK REFERENCE

# The Full HTML Tag Cheat Sheet

| Tag | Category | What it does |
|---|---|---|
| `<html>, <head>, <body>` | Structure | The root containers of every page |
| `<title>, <meta>` | Structure | Page title & invisible metadata |
| `<h1>-<h6>, <p>` | Text | Headings and paragraphs |
| `<div>, <span>` | Text | Generic block / inline containers |
| `<strong>, <em>, <mark>` | Text | Importance, emphasis, highlight |
| `<blockquote>, <code>, <pre>` | Text | Quotes and code formatting |
| `<ul>, <ol>, <li>` | Lists | Bulleted / numbered lists & items |
| `<dl>, <dt>, <dd>` | Lists | Term + definition pairs |
| `<a href="">` | Links | Hyperlinks to pages or sections |
| `<img>, <figure>` | Media | Images with optional captions |
| `<video>, <audio>, <iframe>` | Media | Embedded video, audio, other pages |
| `<table>, <tr>, <th>, <td>` | Tables | Rows, header cells, data cells |
| `<thead>, <tbody>, <tfoot>` | Tables | Grouping table rows |
| `<form>, <input>, <label>` | Forms | Collecting user input |
| `<select>, <textarea>, <button>` | Forms | Dropdowns, text areas, buttons |
| `<header>, <nav>, <main>` | Semantic | Page-level layout regions |
| `<section>, <article>, <aside>, <footer>` | Semantic | Content grouping regions |

SECTION 5

# Local Development Workflow

Recap: how the local server you've already set up fits into this.

RECAP

# Serving Your Page With npx http-server

Why not just double-click the HTML file? Opening a file directly uses the file:// protocol, where some browser features (fetch requests, modules, relative paths) misbehave. A local server serves your files over http://, exactly like a real website.

### `npx http-server`

Run this inside your project folder. It starts a local server and prints the address - usually http://127.0.0.1:8080.

**e.g.** *cd my-project && npx http-server*

### `npx http-server -p 8000`

The -p flag lets you choose a custom port number if 8080 is already busy.

**e.g.** *npx http-server -p 8000*

### `npx http-server -o`

The -o flag automatically opens your default browser to the server's address.

**e.g.** *npx http-server -o*

**The everyday loop**

Edit your .html/.css file -> Save -> Switch to the browser tab -> Refresh -> See the change instantly.

SECTION 6

# Introduction to CSS

Now that the structure is in place, let's make it look good.

CONCEPT

# What is CSS, and Why It Matters

### CSS = Cascading Style Sheets

CSS describes HOW HTML elements should look: colors, fonts, spacing, sizing, and how they're arranged on screen. 'Cascading' means rules can flow down and combine from multiple sources.

- Keeps content (HTML) separate from appearance (CSS)
- One stylesheet can style hundreds of pages consistently
- Makes redesigns fast - change one file, not every page
- Enables responsive design across phones, tablets, desktops

### HTML Without CSS

- Default browser fonts and black text only
- No layout control - everything stacks top to bottom
- No colors, spacing, or visual hierarchy
- Identical, generic look across every site

### HTML With CSS

- Custom fonts, colors, and brand identity
- Flexible layouts - grids, columns, centered content
- Visual hierarchy guides the eye to what matters
- A polished, professional, responsive page

# Three Ways to Apply CSS

## 1. Inline CSS

```
<p style="color: red;">Hi</p>
```

Written directly inside an element's style attribute. Quick for a one-off test, but hard to maintain and impossible to reuse across elements. Avoid in real projects.

## 2. Internal CSS

```
<style>
  p { color: red; }
</style>
```

A <style> block placed inside the <head>. Keeps CSS in one place on a single page - fine for quick prototypes, but doesn't share styles across multiple pages.

## 3. External CSS - Best Practice

```
<link rel="stylesheet" href="style.css">
```

A separate .css file linked from every HTML page. Keeps HTML clean, lets the browser cache the file, and one change updates the whole site at once. This is what professional teams use.

# CSS Selectors - How CSS Finds Elements

```
selector { property: value; }     e.g.    p { color: navy; }
```

### p { }

Element selector - targets every element of that tag on the page.

### .card { }

Class selector - targets every element carrying class="card". Reusable across many elements.

### #header { }

ID selector - targets exactly one element with that unique id.

### h1, h2, h3 { }

Group selector - applies the same rule to several selectors at once, separated by commas.

### nav a { }

Descendant selector - targets <a> elements that live inside a <nav>, anywhere within it.

### a:hover { }

Pseudo-class - targets an element in a particular state, like being hovered, focused, or first-child.

CORE MENTAL MODEL

# The CSS Box Model

### `content`

The actual text, image, or element itself. Sized with width and height.

### `padding`

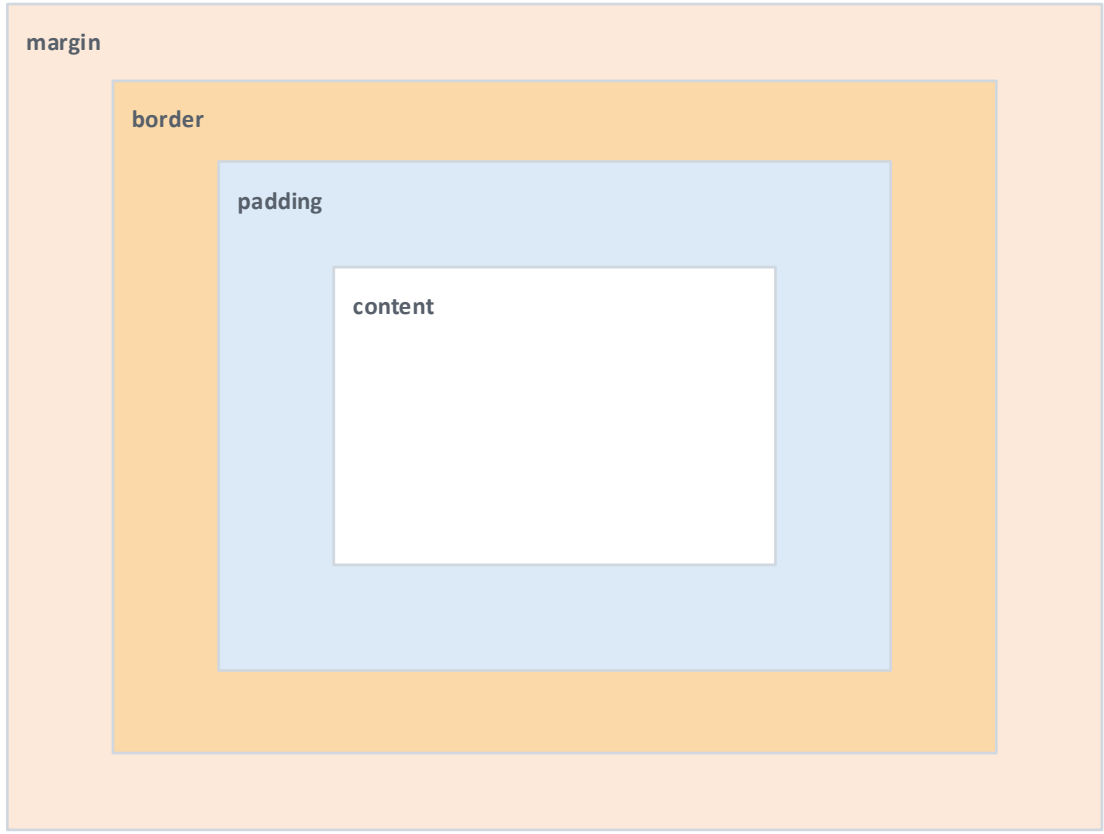
Space INSIDE the border, between the border and the content. Pushes content inward.

### `border`

A visible (or invisible) line drawn around the padding. Set with border-width, -style, -color.

### `margin`

Space OUTSIDE the border, pushing this box away from its neighbours.



# Color, Background & Typography

### color

```
color: #1B1F23;
```

Sets the text color of an element.

### background-color

```
background-color: #F6F8FA;
```

Sets the solid fill color behind an element.

### background-image

```
background-image: url('bg.jpg');
```

Sets an image as an element's background, often paired with background-size: cover.

### font-family

```
font-family: Arial, sans-serif;
```

Sets the typeface. Always include a fallback in case the first choice isn't available.

### font-size

```
font-size: 16px;
```

Sets how large the text renders, typically in px, em, or rem.

### font-weight

```
font-weight: 700;
```

Sets text thickness: normal, bold, or a number from 100 (thin) to 900 (heavy).

PROPERTY REFERENCE

# Spacing, Display & Position

### margin

```
margin: 10px 20px;
```

Space OUTSIDE an element's border, pushing other elements away from it.

### padding

```
padding: 16px;
```

Space INSIDE an element's border, between the border and its content.

### width / height

```
width: 320px; height: 200px;
```

Sets the size of an element's content box.

### display

```
display: flex;
```

Controls how an element lays out: block, inline, inline-block, flex, grid, or none (hidden).

### position

```
position: sticky; top: 0;
```

Controls how an element is positioned: static (default), relative, absolute, fixed, or sticky.

### border-radius

```
border-radius: 8px;
```

Rounds the corners of an element's border box - from subtle curves to full circles.

# Flexbox Basics

### `display: flex;`

Turns an element into a flex container - its direct children become flex items arranged in a row by default.

### `flex-direction`

row (default) lays items left-to-right; column stacks them top-to-bottom.

### `justify-content`

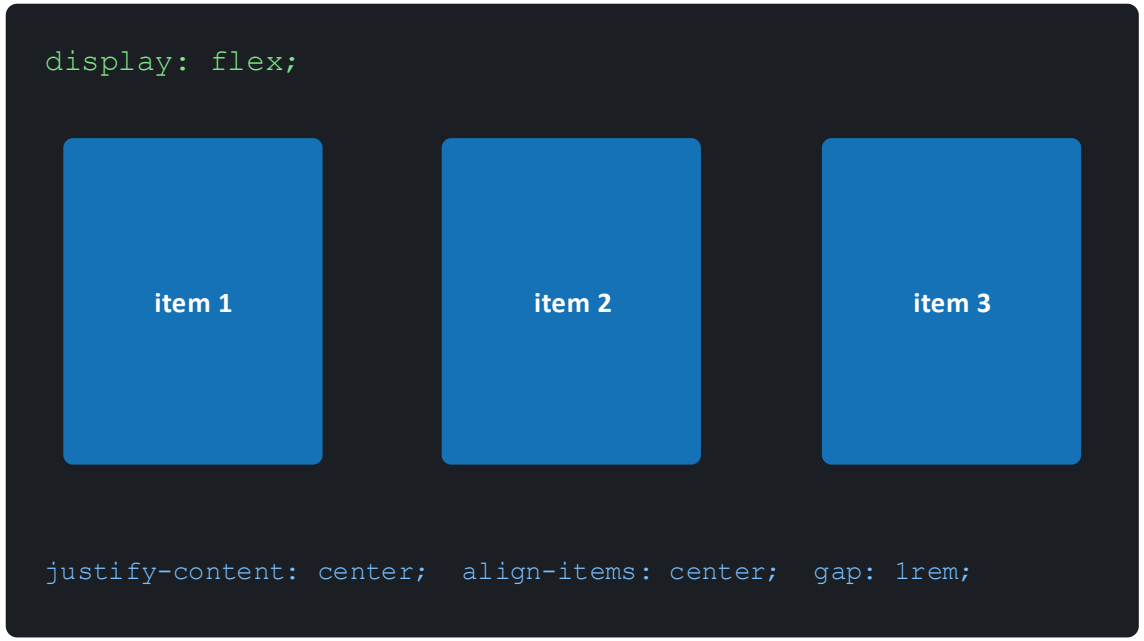
Aligns items along the main axis: flex-start, center, space-between, space-around.

### `align-items`

Aligns items along the cross axis: stretch (default), center, flex-start, flex-end.

### `gap`

Adds consistent spacing between flex items, without needing margin tricks.



# Best Practices for Efficient Styling

### Use external stylesheets

Keep CSS in its own .css file, not inline - cleaner HTML and reusable across pages.

### Prefer classes over IDs

Classes are reusable and have lower specificity; save IDs for unique anchors or JS hooks.

### Use a naming convention

Stay consistent - kebab-case (.card-title) or BEM (.card__title) so any teammate understands instantly.

### Avoid !important

It overrides the normal cascade and makes future fixes harder. Fix specificity instead of forcing it.

### Use shorthand properties

margin: 10px 20px; instead of four separate margin-top/right/bottom/left lines.

### Design mobile-first

Style for small screens first, then add @media queries to enhance for larger ones.

# Importing & Using External Fonts

### 1 · Hosted (Google Fonts)

Link the font in your HTML, or @import it at the top of your CSS.

```
<!-- in your HTML <head> -->
<link rel="stylesheet"
  href="https://fonts.googleapis.com
  /css2?family=Roboto&display=swap">

/* …or at the top of your CSS */
@import url('…/css2?family=Roboto');
```

### 2 · Self-hosted (@font-face)

Download .woff2 into your project, then declare it once in CSS.

```
/* put MyFont.woff2 in /fonts */
@font-face {
  font-family: 'MyFont';
  src: url('fonts/MyFont.woff2')
       format('woff2');
  font-display: swap;
}
```

### 3 · Apply it anywhere in your CSS

```
body {
  font-family: 'Roboto', 'MyFont',
               sans-serif;
}
```

Tip: list the imported font first, then a similar fallback, and always end with a generic family (sans-serif / serif). Use font-display: swap to avoid invisible text while fonts load.

QUICK REFERENCE

# The Full CSS Property Cheat Sheet

| Property | Category | What it does |
|---|---|---|
| `color` | Typography | Text color |
| `font-family / font-size / font-weight` | Typography | Typeface, size, and thickness |
| `text-align` | Typography | Aligns text left, center, right, or justify |
| `background-color / background-image` | Appearance | Fill color or image behind an element |
| `border / border-radius` | Appearance | Outline around an element, optionally rounded |
| `margin` | Spacing | Space outside an element's border |
| `padding` | Spacing | Space inside an element's border |
| `width / height` | Sizing | The size of an element's box |
| `display` | Layout | block, inline, flex, grid, or none |
| `position / top / left` | Layout | How and where an element is positioned |
| `flex-direction / justify-content / align-items` | Flexbox | Arranges flex items in a row or column |
| `gap` | Flexbox / Grid | Spacing between items in a flex or grid container |

SECTION 7

# Practice & What's Next

Time to build something with your own hands, and a look at our upcoming project.

CLASS ACTIVITY

# Hands-On Exercise: Build & Serve a Page

Use the tags and properties from today. Raise your hand at any step if something doesn't look right.

1. Create index.html with the standard boilerplate (doctype, html, head, body)
2. Add a <header> with a heading and a <nav> with 3 links
3. Add a <main> section with a paragraph, an image, and a list
4. Add a simple <form> with a text input, an email input, and a submit button
5. Create style.css and link it with <link rel="stylesheet">
6. Style the page: colors, fonts, spacing, and a flex layout for the nav
7. Run npx http-server in the folder and view the result in your browser

COMING UP NEXT

# Project Preview: The Project Estimate Tool

This is the real project we'll build together over the next few classes, applying everything from today.

### What you'll build

- A form to enter project details: name, task list, hours, and rate per hour
- A table that lists each task with its hours and computed cost
- A styled summary card showing the total estimated cost
- A clean, professional layout using semantic HTML and a flex-based design

```
project-estimate/
  ├── index.html        (form + estimate table)
  ├── style.css          (layout & visual design)
  └── script.js          (calculations - coming with JS)
```

**Skills this project ties together:**

- HTML forms & tables
- Semantic layout tags
- CSS box model & flexbox
- Efficient, reusable CSS classes

KEY TAKEAWAYS

# HTML & CSS in One Slide

### HTML gives meaning, not looks

Tags describe structure & content; let CSS handle appearance

### Use semantic tags

header/nav/main/section/footer help browsers, SEO, and accessibility

### Serve locally while you build

npx http-server avoids file:// quirks and mirrors a real website

### External CSS is best practice

One linked stylesheet keeps HTML clean and styles consistent

### Master the box model

content → padding → border → margin controls all spacing

### Flexbox for modern layout

display: flex + justify-content/align-items solves most layouts

# Thank You

Questions? Let's open the editor, start the server, and build something.

*Terralogic Academy • Batch 3 • MERN Stack Program*